

PASADHI · SOLUTION PAPER

PASADHI: a matrix-first pool built from the machines you already own

Solution paper · 2 figures

PASADHI contributors · v1.1

23 October 2026 · MIT for the open core

github.com/camponggogo/pasadhi · pasadhi.tech

Solution paper · v1.1 · 23 October 2026 · PASADHI contributors · MIT for the open core

Currency: All prices and monetary amounts are in US dollars ($). Original local-currency amounts are divided by 33.65, using the Bank of Thailand reference rate for 22 October 2026. This is a reporting-currency conversion; model assumptions, margins and performance ratios are unchanged.

Abstract

Organisations already own the hardware they need; what they lack is a way to use it as one machine. PASADHI turns office PCs, servers, laptops and small GPUs into a single compute pool with four properties: it is **matrix-first**, **layered as processes**, **priced by measured impact**, and **honest about the physics**.

The paper states the coordination problem formally, derives the three ceilings that decide the architecture, then reports what a 50-machine test bed actually did: 15.7 req/s from one gateway worker and 22.3–24.8 req/s from two; a content-addressed prefix cache that lifts time-to-first-token on an 8 000-token document from 111.4 s to 0.71 s (157×); a document-aware router at 90.15 % accuracy in 31.5 µs; a separated node plane that raises heartbeat capacity from 44 to 161 req/s per shard; and a mixed fleet of 1 450 cards (2 GB to 40 GB) that aggregates 9.54 PFLOPS f32 and 47.7 POPS int8 inside 143.5 kW.

The design rule that survives every measurement is simple: **the inner loop stays local, and the price follows the measurement**. Installation is four moves on every supported OS, shown isometrically in §6.

1. The problem, stated exactly

1.1 Three quantities, one budget

Let a pool serve requests with arrival rate λ and mean service time S. Little's law fixes the number in system, $L = \lambda W$. Every request therefore spends $W = S + Q$ seconds, where Q is time spent waiting rather than computing. The useful fraction of the pool is

```
efficiency  η = S / (S + Q)          utilisation  ρ = λ·S / m        (m = parallel servers)
```

Two regimes follow. When ρ < 0.7 the system is compute-bound and the only thing that helps is more or faster servers. When ρ → 1 the system is **coordination-bound**: adding servers adds queueing faster than it adds throughput, because every new server also adds a control message, a cache entry and a scheduling decision.

The first 50-machine run landed in the second regime. Throughput sat at 12 req/s while latency grew linearly, and at 50 concurrent clients **96.5 % of response time was queueing** on top of a 174 ms service floor. The fleet was not slow; the control path was. This is the problem the paper solves.

1.2 Why the obvious answers fail

Obvious answer	Why it fails here	What we measured
Add more workers per host	one event loop, one accept queue; the host becomes the server	4 workers peak at 29.1 req/s, then fall to 20.5
Add more ports	ports buy isolation, not throughput	2 ports beat 2 workers by 16 %; a userspace proxy gives it back (−25 %)
Shard the model across all machines	activation traffic grows with depth	50-way pipeline ⇒ 65.5 Mbit/s, 1 922× gateway traffic
Put the scheduler in PostgreSQL	the hot path inherits an RPC and a lock	4.5 req/s with errors, versus 15.0 on the local path
Price by core-hour	double-counts idle cores and slow disks	weights must come from the impact sweep, not from core count

2. Background and how this differs

Distributed inference is usually built either as a **pipeline** (Petals-style layer sharding), as a **cluster scheduler** (Ray, Dask, Kubernetes), or as a **volunteer network** (BOINC, Folding@home). Each solves a piece:

- pipelines move activations between stages, which is the right shape for one model that does not fit;
- cluster schedulers place jobs on homogeneous nodes with a fast fabric;
- volunteer networks aggregate embarrassingly parallel work with untrusted peers.

PASADHI targets a fourth case: **an office that owns 100–200 heterogeneous machines on a 1 GbE LAN, wants to run many small-to-medium matrix jobs, and cannot ask staff to babysit a cluster.** That case needs the opposite of a deep pipeline: shallow stages, local inner loops, a control plane that survives heartbeat storms, and a pricing rule a finance team can audit.

Three design commitments distinguish it from the systems above:

1. **Matrix-first.** The unit of work is a matrix operation, not a model layer. GEMM, int8 GEMM, FFT and convolution all reduce to the same tiled kernel, so one scheduler serves many workloads.
2. **Process-per-layer with bounded queues.** Back-pressure is the scheduling signal; no layer can stall another.
3. **Impact pricing.** A contributor is paid by measured effect on delivered work, not by core-hours offered.

3. The architecture in one picture

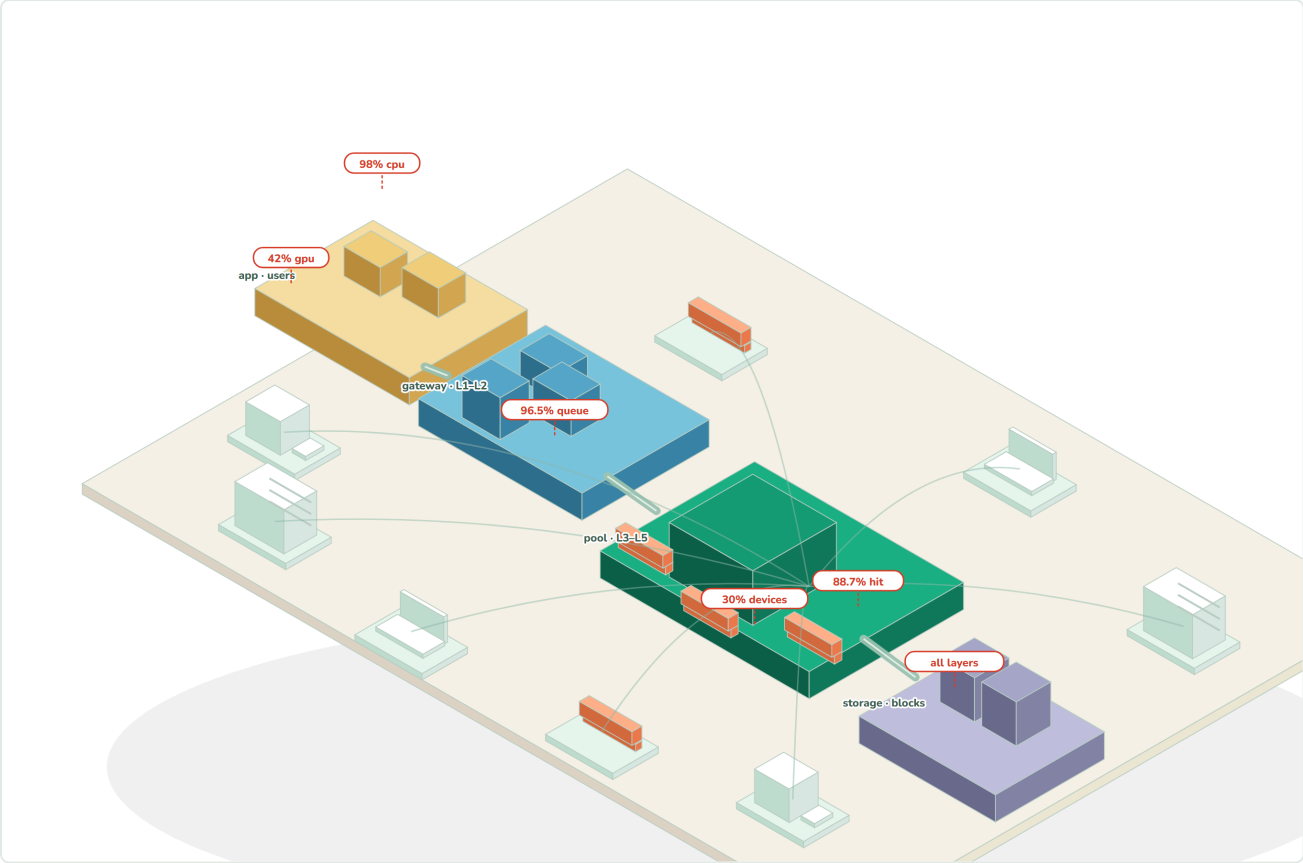


Figure 1. The whole system on one floor. Office devices (bottom and sides) feed the **pool** platform, which holds the tiled matrix kernel and the card inventory; the **gateway** platform holds identity, policy and audit; the **app** platform is what a user touches; **storage** keeps content-addressed blocks. Red tags are the measured shares that drive the design: 30 % of a machine is the usual contribution, 88.7 % of prefixes hit the cache, 96.5 % of response time is queueing at C=50, and the impact weights are 98 % CPU / 42 % GPU.

```
 L5 storage     content-addressed blocks, local disks
L4 kernel       GEMM · int8 GEMM · FFT/conv as GEMM
L3 scheduler    tiling, placement, back-pressure, retries
L2 policy       identity, scopes, budgets, audit
L1 transport    framed TCP; shared memory on the same host
```

Each layer is a process group with its own bounded queue. A slow layer blocks its producer, which is the signal the scheduler needs. The gateway and the node plane are separate processes, so a heartbeat storm cannot delay an inference request; the node list is cached for 5 s, and heartbeats run every 10 s.

4. Principles, and the measurements that produced them

4.1 Serve the median first

Prefix reuse shortens the median dramatically and leaves the tail alone: 157× on the median of an 8 000-token document, p95 unchanged. The cache is therefore a *median* instrument; tail latency must be bought with redundancy, not with cache.

4.2 Scale workers until the host, then stop

The gateway ceiling is the event loop: a raw health endpoint serves 58.7–71.2 req/s, while a completion costs ≈60 ms of loop time. Two workers reach 22.3/24.8 req/s; four workers reach 29.1 at C=8 and fall to 20.5 at C=32. The crossover is the operating point, and it is measurable in ten minutes.

4.3 Separate planes, bound the queues

Moving the node plane out of the gateway raised heartbeat capacity from 44 to 161 req/s per shard and cut C=1 latency from 210 ms to 105 ms. The mechanism is queue isolation, not faster code: the scheduler can now drop stale heartbeats without touching a request.

4.4 Keep the inner loop local

Sharded GEMM keeps only 6.9 % of compute across an antipodal fibre link, 32.3 % over LEO, 2.8 % over GEO and 0.004 % across Earth–Mars. The rule that follows is not "never distribute" but "distribute the outer loop": tiles that fit one host stay on one host, and only the panel boundary crosses the wire.

4.5 Price by impact, not by cores

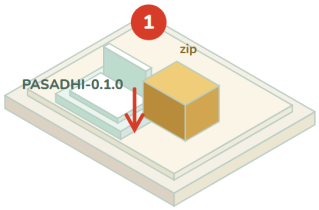
Weights come from the same sweep that produced the performance numbers: CPU 98.0 %, RAM 0.4 %, disk 1.0 %, network 0.6 % on a CPU-only node; CPU 56.6 %, GPU 42.2 % once a GTX 1050 is present. A four-thread machine shared at 100 % earns $0.0743 per hour; an eight-thread machine at 35 % earns ≈$21.13 per month.

5. What we measured, and what it bought

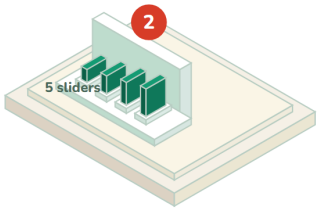
Change	Before	After
Node plane separated from the gateway	44 req/s heartbeat, 210 ms at C=1	**161 req/s per shard, 105 ms**
Live node list cached for 5 s	4.9 req/s at C=8	**15.0 req/s**
Two gateway workers	15.7 req/s	**22.3 / 24.8 req/s** (C=8 / C=16)
Prefix cache, 8 000-token document	111.4 s to first token	**0.71 s, 88.2 % reuse**
Document-aware router	—	**90.15 %, 31.5 µs**
Domain experts instead of a mixture	86 machines, infeasible	**32 machines, 0.64 req/s (10.7×)**
Mixed fleet of 1 450 cards	—	**9.54 PFLOPS f32, 47.7 POPS, 143.5 kW**

Three numbers decide architecture rather than tuning: the loop ceiling (58.7–71.2 req/s raw), the cache win (157× median), and the port illusion (100 ports = 4.4 GiB of memory, no extra request per second).

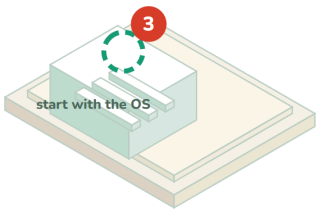
6. Installing it, in four moves



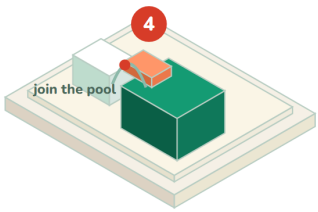
download the build



probe, then set the five shares



let it start with the OS



it joins the pool and earns

Figure 2. Installation as it actually runs, in isometric order: (1) unpack the build, (2) probe the machine and set the five shares, (3) let it start with the operating system, (4) join the pool and begin earning.

Step	Windows	Linux	macOS	Android
1 · unpack	`PASADHI-0.1.0-win64-portable.zip`	tarball	`PASADHI-0.1.0-macos.tar.gz`	APK
2 · probe	`python service.py --once`	same	same	in-app probe
3 · autostart	`install.py --autostart` (Run key)	systemd `--user` unit	LaunchAgent	foreground service
4 · join	open `127.0.0.1:8765`, set shares	same	same	same

The probe reports cores, RAM ceiling, disk free and network class, then proposes a share. The five sliders — CPU, GPU, RAM, disk, network — are the only configuration a contributor ever sees; the worker score and the live monthly estimate update as they move. Nothing needs a reboot, and `uninstall.py` removes every trace.

7. Capacity and the commercial model

At a 30 % share a 150-machine office delivers 3 564 GFLOP/s and 31 860 node-hours per month, which earns $1,588.71 against a $4,450.07 subscription: the break-even is ≈60 % share, or ≈$10.40 per machine. The product is therefore sold three ways — **share and use free**, **rent monthly or yearly**, **buy an on-prem licence** scoped to a group with unlimited machines.

At fleet scale the same arithmetic holds: 1 450 machines, 68.1 % gross margin, 11.5 % of revenue issued as credits, and a stress case at 720 h per machine that still clears 43.5 %.

8. What is deliberately not in the product

- **No 50-way pipeline sharding** — S* is 3–10 stages, overhead is 150 ms per step, activation traffic 65.5 Mbit/s.
- **No interplanetary inner loop** — 0.004 % of compute survives Earth–Mars; only checkpointed work crosses it.
- **No claim of cheaper FLOPs than a GPU** — a used-card fleet lands at $0.00002972 per delivered TFLOP, the same as a rented A100. The margin comes from software, support and residency.

9. Threats to validity

Every measurement here comes from one host (8 vCPU, Docker Desktop), a mock backend and published card specifications. Absolute ceilings move with the host; the shape of the results does not. Fleet numbers are models parameterised by measured per-card throughput, not a 1 450-machine deployment. Where a figure is modelled rather than measured, the text says so. The full evidence table, with every graph and every run, is the companion volume *PASADHI Full Paper*.

10. Roadmap

Milestone	Evidence it is done
M1 packed int8 kernel in C/AVX2	25–45 GFLOP/s on the reference part, ≤1.5 % error
M2 shared-memory transport	pipeline ≥1.5× the single loop at 512^2 blocks
M2.7 group controller	join tokens, quotas, offline autonomy, per-department billing
M3 distributed matrix plane	row-panel sharding, retry-safe tiles, recovery after a mid-panel kill
M6 public network	redundant execution 2× + 5 % spot checks, credit ledger with a daily Merkle root

Appendix: where each number lives

`report/RPS_LIMIT.md` (gateway ceiling), `report/MULTIPORT_100.md` (ports), `report/KV_CACHE.md` (prefix reuse), `report/EXPERTS_WHITEPAPER.md` (router), `report/HETERO_FLEET.md` (mixed fleet), `report/IMPACT_PRICING.th.md` (weights), `report/GROUP.th.md` (group economics), `pasadhi/core/orbit.py` (link budgets), `report/IMAGE_FACTS.md` + `report/IMAGE_FACTS_KERNELS.md` + `report/IMAGE_FACTS_MODELS.md` (what every figure shows).